Python come calcolatrice intelligente

L1 · Informatica per economisti · SIMA 2026

Oggi

1

Spiegazione e demo

25 min

Programmi, Colab, Gemini, errori

2

Task

60 min

Il risparmio di Marco, a piccoli passi

3

Debrief

25 min

Variabili, tipi, la prima trappola

Cos'è un programma

Una lista di istruzioni, eseguite in ordine, una dopo l'altra.

Il computer non capisce: esegue. Esattamente quello che scrivi.

PYTHON

```
capitale = 5000
tasso = 0.025
print(capitale * (1 + tasso))
```

OUTPUT

```
5125.0
```

Colab: il nostro ambiente

Nel browser
Gratis, niente da installare

Account Google
L'unica cosa che serve

Celle
Testo e codice, alternati

Shift + Invio
Esegue la cella selezionata

Demo apriamo il notebook dal pulsante «Open in Colab» nel README del corso

Gemini: l'assistente

1. Chiedi cosa vuoi, con precisione
2. Leggi il codice che propone
3. Verifica il risultato

REGOLA DEL CORSO

Che il codice giri non vuol dire che sia giusto.

Demo «calcola l'interesse composto su 1000 euro al 3% per 10 anni»

Gli errori sono amici

PYTHON

```
print(capitale
```

ERRORE

```
SyntaxError: '(' was never closed
```

1

Leggi l'ultima riga

dice il tipo di errore e la causa

2

Copiala su Google

così com'è, senza modificarla

3

Qualcuno l'ha già risolto

di solito su Stack Overflow

Come si lavora nel task



Passi piccoli

Uno alla volta, nell'ordine



Cosa cercare

Per ogni passo trovi le parole chiave



Gemini c'è

Ma prima prova a capire cosa ti serve



10 minuti

Bloccato più a lungo? Chiedi

Saper cercare è metà del mestiere di programmare.

Se siete bloccati



Durante la lezione

Scrivete nella chat di Zoom, o usate «Chiedi aiuto» nella stanza: entro io.



A casa

In Colab: Condividi → aggiungete il mio indirizzo (è nella mail) come Editor. Poi scrivetemi quale passo non torna.

Con il permesso di Editor posso leggere il vostro codice e correggerlo direttamente nel vostro notebook.

In gruppi da 3, con tre ruoli



Chi scrive

condivide lo schermo e scrive il codice nel suo notebook



Chi cerca

Google, Gemini, dispense: trova cosa serve



Chi controlla

il risultato ha senso? il numero torna?

I ruoli ruotano a ogni incontro. Il più importante è chi controlla: è il mestiere che il corso vuole allenare.

Task: il risparmio di Marco

Marco ha 5.000 € e li investe al 2,5% annuo.

60 minuti · apri il notebook dal README

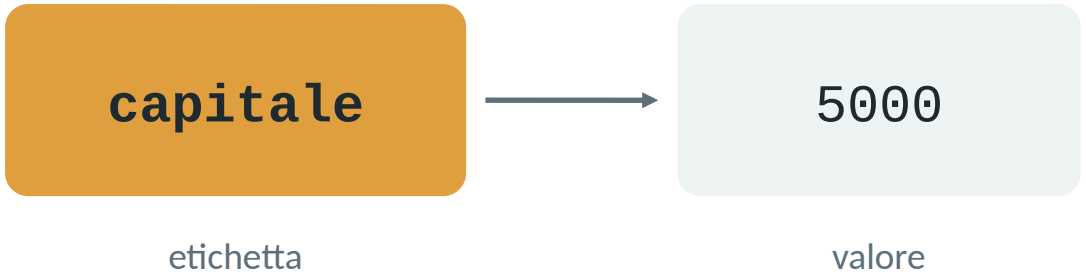
Debrief

Come ha funzionato, e perché

Variabili

PYTHON

```
capitale = 5000
```



- Un'etichetta attaccata a un valore
- Il valore si può cambiare quando vuoi
- Il nome lo scegli tu: meglio se dice cosa contiene
- = non è «uguale»: è «metti questo valore qui»

= mette, == chiede

= assegnazione

un ordine: metti questo valore in questa etichetta

PYTHON

```
montante = 6400
```

OUTPUT

```
(nessun output: montante ora vale 6400)
```

== confronto

una domanda: queste due cose sono uguali?

PYTHON

```
montante == 6400
```

OUTPUT

```
False
```

Altri confronti: `>` `<` `>=` `<=` `!=` — rispondono sempre True o False (tipo bool). Scambiare = con == non dà errore: cambia il valore in silenzio.

Tipi di dato

Tipo	Esempio	Cos'è
`int`	`5000`	numero intero
`float`	`0.025`	numero decimale
`str`	`"5000"`	testo, tra virgolette
`bool`	`True`	vero o falso

ATTENZIONE

I decimali si scrivono con il punto

`0.025` e non `0,025`

`type(x)` ti dice il tipo di x

La trappola del passo 7

PYTHON

```
capitale_testo = "5000"
print(capitale_testo * 2)
```

OUTPUT

```
50005000
```

Nessun errore.
Risultato sbagliato.

Python ha ripetuto il testo due volte. Il codice che gira non è per forza giusto: è la lezione più importante del corso.

Testo più numero

PYTHON

```
print("Il capitale è " + capitale)
```

ERRORE

```
TypeError: can only concatenate
str (not "int") to str
```

Due modi per correggere

PYTHON

```
"Il capitale è " + str(capitale)
```

PYTHON

```
f"Il capitale è {capitale}"
```

Python non indovina: testo e numeri vanno trattati in modo esplicito.

Extra: una copia del corso tutta tua

1

Crea un account

su github.com, gratuito

2

Apri il repository

github.com/andrealorenzon/SIMA2026

3

Clicca «Fork»

in alto a destra, poi «Create fork»

4

È tuo

github.com/<tuo-nome>/SIMA2026

- Un fork è una copia del repository nel tuo account: puoi modificarla senza toccare l'originale.
- Da Colab salvi i notebook direttamente lì: File → Salva una copia in GitHub.
- Quando il corso si aggiorna, sul tuo fork clicca «Sync fork» per ricevere le novità.

Prossima volta: un file Excel

Compito: Kaggle Learn – Python, lezione «Hello, Python» (~30 min)